

SRW3 Phase 1B — Generalized State, Effects, Aliasing, and Authority

Phase: 1B — CM2 resource aliasing + CM3 aggregate/delegated authority + generalized finite-effect transitions

Status: COMPLETE. The commitment-gated safety calculus survives both open countermodel classes **without any ad hoc special case**. The two CM2/CM3 counterexamples are formalized, executable, and permanent regressions; the alias-aware and aggregate-authority lineage-length safety theorems ($\text{InitialSafe}_* \wedge \text{SecurityClosed}_* \rightarrow \forall n. \text{Safe}_*$) are **PROVED BY K** over the generalized semantics (GH2-T2/M, GH3-T2/M, GH4-T2/M all reach #Top via [circularity], with all three non-vacuity negative controls failing exactly as designed); the generalized gate evaluates aggregate authority **as one more obligation class inside the existing one-gate architecture** (Part VI architecture claim demonstrated); a Python mirror oracle reproduces the K decision table 14/14; an additive KEVM binding extension executes CM2/CM3-shaped obligation sets over REAL EVM storage with byte-identical carriers across the haskell and llvm backends; full regression matrix v5 shows zero change on the entire Phase-1A surface.

Deliverables: k/srw3gen.k (generalized semantics, 641 lines), k/gen-demos/ (10 demos + self-checking runner), proofs/gen/{cm2_alias,cm3_auth,gen_full}.k + ghost_manifest_gen.json (three ghost proof artifacts with fidelity manifest), python-gen/ (mirror oracle + 14 tests), k/kevm/srw3-gen-binding.k + 2 generalized EVM demos (additive binding extension), audit transcripts under transcripts/audit/ (incl. full_matrix_v5.txt), this report (MD + PDF).

1. Starting state and scope

Phase 1A froze the LLVM retarget as the verification baseline: the abstract demo suite (13 demos), Claims 1–4b, the ghost-state $\forall n$ safety theorem over the ghost-instrumented GateRun semantics (GH-T2/GH-M), the Tier-1/Tier-2 reconciliation, the KEVM binding, and the haskell $\leftrightarrow$ llvm semantic-equivalence surface. Three boundaries were explicitly recorded as not yet mechanized: the Tier-3 universal statement, the cryptographic lineage binding, and — the subject of this phase — the two open countermodel classes CM2 (resource aliasing) and CM3 (aggregate/delegated authority).

The Phase-1B question is whether the SRW3 commitment-gated calculus can absorb these two classes **and** generalize from the single-write GateRun abstraction to finite multi-write, multi-resource transitions, without weakening any invariant, adding a second gate mechanism, or introducing ad hoc special cases. Every extension must therefore enter the calculus through the established doors: evaluated obligations, declared-footprint applicability, and the single accept rule that updates the committed state.

The baseline for this phase was frozen first (Part I): the Phase-1A work products were committed as phase1a-final = 88e4209 with a provenance record at transcripts/provenance/phase1b_baseline.md, the complete Phase-1A regression matrix (11 stages) was re-executed in the foreground with unchanged results, and a clean phase1b-start branch was cut before any Phase-1B source modification.

2. Environment note: sandbox reset and recovery

Mid-phase, the sandbox was reset and the srw3-work/ repository directory was lost (the working area is /home/z/my-project/srw3-kevm/, which survived as plain files; the phase-1A git objects did not). All Phase-1B semantic work products survived and were verified byte-exactly against the recorded fidelity manifest before any resumption: k/srw3gen.k sha256 d583ef0c... matches ghost_manifest_gen.json's base_sha256, so every downstream proof artifact built on it remains valid

evidence.

The pinned toolchain was reinstalled from the Phase-0-D recipe and re-verified: K v7.1.337 (jammy .deb, extracted), z3 4.13.3, flex/libfl2 2.6.4, libsecp256k1 0.5.0 (with the soname-compatible symlinks), libllvm15 + llvm-15/clang-15/lld-15 15.0.6-4+b1 (bookworm pool) for the LLVM link chain, and the KEVM v1.0.921 source with blockchain-k-plugin K sources re-fetched at the pinned SHA 207ae512... ldd on the kore binaries matches the Phase-0-D \$A library list exactly. Two toolchain lessons were recorded for the report: (a) the sandbox reaps **all** background processes between tool invocations (a setsid nohup sentinel died within seconds), so every long-running verification must fit foreground chunks — the per-claim proof runner was made resumable instead; (b) the LLVMKompilePrelude/llvm-kompile-clang chain hard-codes toolchain paths that must be reprinted for a no-root extraction (same deviations class as Phase-1A §3, re-applied and re-documented).

3. CM2 formalization: physical keys vs logical resources (Part II)

CM2 is the observation that physical storage identity and logical resource identity can come apart: two distinct storage keys can resolve to the same logical resource, and a system that reasons only about physical key footprints will under-approximate the logical effects of a write through an alias. The generalized semantics makes this distinction explicit with three additions to the syntax: a finite sort `ResourceId`, an external resolution table `resmap : StKey → ResourceId` (the model input `Resolves`), and the requirement that the abstract counterexample be representable: `(appA, slot7)` and `(appB, slot3)` are distinct `StKeys` while `Resolves(K1) = Resolves(K2) = resP`.

The counterexample was constructed **before** any defense, as the mandate requires. Program `cm2_counterexample.srw3`: A initializes `stK(appA, 7) ← 5`; B writes `stK(appB, 3) ← 6` through the alias; the logical resource invariant IRP requires `stK(appA, 7) + stK(appB, 3) ≤ 10`. Under the naive registry (`aliasNaive`), the resource obligation IRP is app-anchored at the primary holder only, so B's aliased write raises **no** obligation: the gate commits, and the committed state violates IRP ($5 + 6 = 11 > 10$). The purely-physical footprint system is shown failing on its own terms — the true logical effect set is `{resP}`, the declared physical footprint is `{stK(appB, 3)}`, and the missed alias obligation is exactly the gap between them. This demo is a permanent regression:

`test_cm2_counterexample_naive_commits_alias_unsafe` in the Python oracle asserts both the commit decision **and** the violated invariant on the committed state.

4. Alias-aware effect semantics (Part III)

The defense was derived from the counterexample rather than chosen from a catalog. The effect model normalizes physical footprints to logical resources by the closure `AliasClosure(E) = { r | ∃k ∈ E. Resolves(k) = r }`, realized as the total function `logicalOf(F, resmap)`; obligation applicability is then evaluated per scope: app-scope edges fire on the app projection `appsOf(F)` (Phase-0 behavior, unchanged), **resource-scope edges** `edgeR(J, Rs)` **fire on** `logicalOf(F, resmap)`, and authority-scope edges (Part V/VI) fire on `authOf(F)`. Completeness keeps its Phase-0 form at the physical granularity (`wfCheck` enforces $FT ⊆ FD$ under `enforce`); the logical-level statement follows by monotonicity — under the physical premise, `AliasClosure(FT) ⊆ AliasClosure(FD)`, so applicability computed from the declared footprint coincides with the true one. The strengthened completeness condition under which CM2-style mis-decisions become impossible is therefore: every resource obligation is covered by an `edgeR` whose scope contains its resource dependencies (`resourceComplete`), and applicability is evaluated over the alias closure of the declared footprint.

Two deliberate non-choices are recorded. First, the alias relation itself (`resmap`) is an **external model input** — the calculus consumes the resolution table as data and enforces obligations uniformly over its closure, but does not derive which keys alias which resources; this is assumption A5, classified

ASSUMED and stated in the header of `srw3gen.k` (a wrong or incomplete `resmap` is a modeling error outside the calculus). Second, no axiom asserts "the alias relation is sound"; the theorems below are conditional on the closure being the model's declared input, which keeps the proof surface honest.

5. CM3 formalization: membership is not aggregate safety (Part V)

CM3 is the observation that authorization-closure membership checks compose badly under delegation: individual grants can each be within their own bound while their sum exceeds the root budget. The generalized semantics adds a minimal bounded-delegation representation — `cap : AppId → Int` (granted capacity), `parent : AppId → AppId` (delegation tree), and consumed usage `usage-c/usage-p` threaded through the commitment boundary — with the aggregate predicate stated as an evaluated obligation `IAGG: usage(appC) + usage(appD) ≤ cap(appB)`, over the individual-capacity obligations `ICAPC/ICAPD`. The representation is deliberately minimal: nothing in the semantics hard-codes a governance model; the topology (both delegates under root `appB`) is the concrete instance used by the theorem instances and demos.

The counterexample again preceded the defense. Program `cm3_overflow_counterexample.srw3`: root grants budget 10; delegates are granted 6 and 6; the first transition consumes 6 (within every individual bound, gate commits), the second consumes 6 (each membership check `ICAPC/ICAPD` evaluated in isolation passes). Under the naive registry (`authNaive`), the gate commits the second transition and the committed usage state violates the aggregate budget: $6 + 6 = 12 > 10$. This establishes by executable witness that `AuthorizationMembership ≠ AggregateAuthoritySafety` — the permanent regression asserts both the naive commit and the violated aggregate invariant in the Python oracle (`test_cm3_overflow_counterexample_naive_commits`, `test_cm3_membership_is_not_aggregate_safety`).

6. The authority-aware commit gate (Part VI)

The scientific test of Part VI is whether CM3 can be absorbed as **another obligation class** without creating a second gate mechanism. It can, and the implementation is exactly four evaluated additions inside the existing machinery: the aggregate invariant `IAGG` is an ordinary evaluated equation over the prospective usage bundle; the authority-scope edge `edgeA(IAGG, {appC, appD})` makes it applicable whenever the declared footprint's authority projection touches either delegate (consumption declares an `AppId` item into the footprint, in the same discipline as writes); the obligation universe gains the three authority obligations; and the gate's accept rule continues to require **one** conjunction `allHoldG(applicableGen(registry, FD, resmap), P, UP, CP)` over the prospective bundle. The accept rule remains the only transition that updates `<committed>/<usage-c>`; rejection reasons remain uniform (`reject` on violated obligations, `reject-auth` on failed membership, `blocked-effect-incomplete` on failed `wfCheck`); no "authority gate" branch exists. The K claim `GATE3-AGREE` verifies the architecture claim mechanically: the certificate-predicted accept condition equals the real gate's evaluated applicable-obligation conjunction, with the aggregate obligation evaluated from `cap` — no special branch.

7. The generalized ghost theorems (Parts IV and VII, and the full suite)

The $\forall n$ theorems are mechanized over ghost-instrumented copies of `srw3gen.k`, built by the same additive discipline as Phase 0-D (semantics bytes identical except `//GHOST`-marked lines; fidelity certified mechanically in `ghost_manifest_gen.json`). Each artifact carries an observer pair (`ghostSafe*`, `ghostMatches*`), a certificate transformer applied at the commitment point (`ghostApply*` on accept, `ghostUndo*` on reject), and the claim suite: `BASE/A/R` (obligations and preservation), `SOUND-I/F`

(certificate→real-state bridge), GATE-AGREE (architecture claim), ISO/T1 (non-circular diagnostics), T2 (the $\forall n$ theorem, [circularity]), M (the stronger GhostMatches-at-every-termination form), and NEG (the non-vacuity control, which must fail).

The results, per suite, are: **cm2_alias** (alias-aware model, class tk2): 11 PROVED + GH2-NEG fails as designed — including GH2-T2/GH2-M. **cm3_auth** (aggregate-authority model, classes tc/tD): 11 PROVED + GH3-NEG fails as designed — including **GH3-T2/GH3-M**, the CM3 aggregate-authority lineage-length safety theorem. **gen_full** (single generalized model — multi-write across four accounts, aliased resources, and authority consumption in one calculus, class tG): 11 PROVED + GH4-NEG fails as designed — including **GH4-T2/GH4-M**, the generalized $\forall n$ theorem. Each T2 instance is stated in the mandated form: $\text{InitialSafe}_*(s_0) \wedge \text{SecurityClosed}_*(\text{registry}, \text{enforce}, \dots) \rightarrow \forall n. \text{Safe}_*(s_n)$, with the closure premise evaluated against the concrete registry term (non-vacuous by construction, and NEG demonstrates the premise is load-bearing).

Two proof-engineering defects were found and fixed during this stage, both recorded because they are exactly the failure modes a reviewer must look for in this encoding. (1) **Variable correlation**: the circular claims shared the variable $N1$ between the transition amount $\text{tc}(N1)/\text{tG}(V1, V2, N1)$ and the lineage counter cell $\langle \text{lineageNext} \rangle N1$. The shared variable silently correlates "the amount being consumed in this cycle" with "the lineage index", the circular re-application stalls, and the proof diverges (GH3-T2 exceeded a 2400 s budget; a depth-bounded run exposed the residual $\# \text{Not}(N = N1)$). The cm2 suite — which separates the two (amount $V1$ vs counter $N1$) — closed, confirming the diagnosis. Fix: claims-module-only rename of the amount variable ($NA/NA1/NA2$), theorem statements unchanged (the amount was already implicitly quantified independently); regenerated artifacts, manifest, and verdicts. (2) **Missing concrete-input rules**: the gen_full ghost module referenced the concrete model inputs $\text{rmPQ}()/\text{capG}()/\text{parentG}()$ without defining them, so every transition-touching GH4 claim — including the pure-evaluation GATE4-AGREE — stuck at exit 113. Fix: add the three syntax declarations and defining rules (values identical to the cm2/cm3 instances and the demo runner), recompile, re-run. Both fixes are visible as dated comments in the fragments and in the v2 header of ghost_gen_theorem.txt (v1 transcript preserved as ghost_gen_theorem.v1-sharedvar.txt).

8. Python generalized model (mirror oracle)

The Python oracle `python-gen/srw3gen_model.py` mirrors the K semantics module-for-module (STATE, INVAR with the nine evaluated obligations, CLOSURE with `logicalOf/appsOf/authOf/per-scope` applicableGen/the four completeness predicates, GATE with `begin/w/wH/rd/rdH/consume/wfCheck/gate` and the three uniform outcomes). Its test file asserts the same decision table as the K demo runner (10 demos) plus four countermodel properties: the CM2 closure captures the counterexample (the alias-aware registry fires IRP on the aliased footprint while the naive one does not), membership is not aggregate safety (the ICAPC/ICAPD conjunction holds while IAGG fails on the same usage map), alias-closure monotonicity under the physical `wfCheck` premise ($\text{FT} \subseteq \text{FD} \rightarrow \text{logicalOf}(\text{FT}) \subseteq \text{logicalOf}(\text{FD})$), and closure-completeness of the full registry. 14/14 PASS. The oracle's value is independent reproduction: any future edit to `srw3gen.k` that silently changes a decision will surface as a Python/K divergence in addition to a K-suite failure.

9. KEVM generalized binding (additive extension)

The frozen Phase-1A binding (`k/kevm/srw3-kevm.k`) is **not modified**. The extension module `k/kevm/srw3-gen-binding.k` (SRW3-GEN-EVM, imports the binding) adds the two Phase-1B obligation classes over REAL prospective EVM storage — `#w3IAliasG`: the two delegate accounts' slot-3 storages mirror one logical resource and must agree (the EVM shape of CM2: distinct physical keys, shared logical identity); `#w3IAGgG`: the delegates' slot-2 consumption sum must stay within the root account's slot-2 budget (the EVM shape of CM3) — and defines `#w3GateG`, which keeps the base gate's exact

shape: one evaluated conjunction ($\#w30b1G = IOL \wedge ILD \wedge IOLD \wedge IAliasG \wedge IAggG$) over real storage, same threaded state carrier, same commit/restore machinery. Slots 0/1 continue to carry the Phase-0 price/liquidation data so both obligation generations compose without interfering.

Two demos exercise the extension end-to-end, with verdicts read from the threaded carrier (matrix-v3 convention; krun exit 1 is the designed terminal state of the carrier): `evm_gen_positive` ($b1=30, b2=40, \text{budget } 100, \text{alias pair } 7 == 7$) → **COMMIT** ($n=1, h=0$, full storage snapshot committed, lineage appended); `evm_gen_overflow` ($60 + 50 > 100$) → **REJECT + RESTORE** ($n=0, h=-1$, all committed storages restored). Both demos run under the haskell and llvm backends with **identical** carriers (`transcripts/audit/evm_gen_equivalence.txt`). This demonstrates the Part VI claim at the real-execution layer: CM2/CM3 enter KEVM purely as additional evaluated obligations of the unchanged one-gate architecture.

10. LLVM portability of the generalized layer

The full generalized semantics (`srw3gen.k`) kompiles under the llvm backend (10.0 s after the toolchain reconstruction) and the 10-demo generalized suite runs with the same decision table under both backends; the canonical-cell comparator reports 10/10 SEMANTICALLY EQUIVALENT (result, committed, usage-c). As in Phase 1A, llvm is an execution-layer portability result only: `kprove` remains haskell-bound, the proof boundary is unchanged, and no proof-surface claim is extended by the llvm runs. Combined with §9, the generalized calculus is now demonstrated at all three execution layers: abstract K (both backends), the KEVM binding (both backends), and the independent Python oracle.

11. Regression matrix v5

Matrix v5 (`scripts/audit_matrix_v5.sh → transcripts/audit/full_matrix_v5.txt`) re-executes every Phase-1A stage plus the Phase-1B additions, in the resumable foreground-chunk discipline forced by the environment note (§2). Phase-1A surface, zero change: [1] Python 15/15; [2] abstract demos 13/13; [3] Claims 1-4b #Top; [4] negative control fails (exit 113); [5] reconciliation bridge 9/9; [6] phantom-reject defect audit; [7] SF/GB premise-form results; [8] KEVM carriers exactly canonical (positive COMMIT $n=1$; stale/min2 REJECT+RESTORE $n=0, h=-1$); [9] Phase-0D ghost encoding 12 claims with certificate; [10] tier-3 stuck with statement unweakened. Phase-1B additions, all green: [GEN-1/2] generalized demos 10/10 (haskell) and 10/10 (llvm); [GEN-3] 10/10 semantically equivalent; [GEN-4] live re-run of all 35 generalized ghost claims — 32 PROVED and the three NEG controls correctly failing (per suite 11 PROVED + NEG; `gen_full` has no A1 because the single `tG` class covers both delegates in one cycle); [GEN-5] Python oracle 14/14; [GEN-6] KEVM generalized binding PASS/PASS with `hs↔llvm` identity.

12. Updated evidence matrix

Result	Phase-0/1A	Phase-1B
$\forall n$ safety (single-write ghost model)	PROVED BY K	unchanged
$\forall n$ safety, alias-aware (CM2)	open (CM2)	PROVED BY K (GH2-T2/M)
$\forall n$ safety, aggregate authority (CM3)	open (CM3)	PROVED BY K (GH3-T2/M)
$\forall n$ safety, generalized multi-write + alias + authority	not representable	PROVED BY K (GH4-T2/M)
CM2/CM3 counterexamples	—	formalized + executable; permanent regressions (K demos, Python oracle)
Membership $\neq$ aggregate safety	—	executable witness + GATE3-AGREE (no special branch)

Result	Phase-0/1A	Phase-1B
AliasClosure completeness under physical wfCheck	—	PROVED BY K (GH2 suite) + Python property
Generalized gate on REAL EVM storage	—	DEMONSTRATED BY KEVM (additive SRW3-GEN-EVM; hs $\leftrightarrow$ llvm identical)
Tier-3 universal reconciliation	FORMALIZED; NOT MECHANIZED	unchanged (out of scope this phase)
Cryptographic lineage	NOT YET MECHANIZED	unchanged (out of scope this phase)

The evidence classification is unchanged from Phase 1A in one important respect: the llvm runs add execution portability only, and the proof boundary remains the haskell/Kore prover. The three new PROVED-BY-K rows are upgrades of the abstract-layer theorem surface; the KEVM row is a demonstration row and is labeled as such.

13. Assumptions (cumulative, classified)

Inherited from Phase 0-D: A1 (dependency oracles are auditable manual derivations), A2 (opaque commitments at the abstract layer), A3 (authorization evidence = membership + evaluated obligations; no signatures), A4 (closure premises evaluated against concrete registry terms). Phase 1B adds: **A5 — the alias table `resmap` is an external model input**; the calculus enforces obligations uniformly over its closure but does not derive it; a wrong or incomplete table is a modeling error outside the calculus. **A6 — `cap/parent` (granted capacity and delegation tree) are external model inputs** for the minimal bounded-delegation topology; the aggregate obligation is evaluated from them, not asserted. Both are stated in the header of `srw3gen.k` next to the code that consumes them, and both are classified ASSUMED in the five-level evidence scheme.

14. Limitations and what remains open

The generalized theorems are proven over the abstract ghost-instrumented semantics with the concrete bounded universe used across the project (four accounts, two resources, three authority obligations); the universes are explicit parameters in `securityClosedGenU`, but the mechanized instances are the concrete ones. The alias table and delegation topology are inputs (A5/A6), so the theorems are conditional on their correctness — deriving `resmap` from EVM storage-layout facts, and capacity trees from on-chain grant logs, remain future binding work. Tier-3 universal reconciliation and the cryptographic lineage binding are untouched and retain their Phase-1A classifications. The KEVM generalized demos exercise storage-level obligations; gas-sensitive and reentrancy-shaped obligation classes are not modeled. Finally, the sandbox reset destroyed the phase-1A git objects; Phase-1B provenance is carried by the fidelity manifests, sha-verified recovery, and a fresh self-contained bundle rather than by continuity of git history (§2).

15. Artifact index

Artifact	Role
<code>k/srw3gen.k</code>	Generalized semantics (641 lines): <code>ResourceId/resmap</code> , <code>AliasClosure=logicalOf</code> , per-scope applicability, usage through the boundary, 9-obligation universe, <code>securityClosedGenU</code> , 6 registries
<code>k/gen-demos/</code> + <code>scripts/run_gen_demos.sh</code>	10 generalized demos with self-checking contract table (10/10 hs, 10/10 llvm)

Artifact	Role
proofs/gen/cm2_alias.k, cm3_auth.k, gen_full.k, ghost_manifest_gen.json	Three ghost proof artifacts (fidelity manifest; 35 claims; 32 PROVED + 3 designed-FAIL)
scripts/gen_fragments/, scripts/gen_ghost_proofs.py, scripts/audit_ghost_gen.sh	Fragment sources, artifact generator, resumable per-claim prover
python-gen/srw3gen_model.py, python-gen/test_gen.py	Mirror oracle; 14/14 (10 demos + 4 countermodel properties)
k/kevm/srw3-gen-binding.k, k/kevm/demos/evm_gen_{positive,overflow}.srw3evm	Additive generalized binding + 2 EVM demos (COMMIT / REJECT+RESTORE, $hs \leftrightarrow llvm$ identical)
transcripts/audit/ghost_gen_theorem.txt (+ .v1-sharedvar.txt)	Authoritative per-claim verdicts; v1 preserved with the variable-correlation diagnosis
transcripts/audit/full_matrix_v5.txt (+ _ghostgen.txt)	Regression matrix v5 (Phase-1A stages + GEN-1..6)
transcripts/audit/evm_gen_equivalence.txt, llvm_gen_equivalence.txt	Binding carrier equivalence; abstract-layer $hs \leftrightarrow llvm$ equivalence
transcripts/provenance/phase1b_baseline.md	Part-I freeze record (phase1a-final 88e4209; matrix v4 re-run)